TOPIC 1

Introduction to VLSI Design

Subtopic 1a: VLSI technology and its applications · Subtopic 1b: the VLSI design flow. Tiered Basic→Industry, with worked examples and hands-on tutorials.

- 1a · What VLSI is, IC evolution, Moore's law, applications, ecosystem, ASIC/FPGA/SoC
- 1b · The design flow, front/back-end, RTL-to-GDSII
- Practical examples · Worked example (4-bit counter) · Tutorials T1–T2 · Glossary

TOPIC 1A · OVERVIEW

What Is VLSI? From a Transistor to a Billion of Them

VLSI (Very-Large-Scale Integration) is the practice of fabricating an entire electronic system — millions to billions of transistors — onto a single piece of silicon called a die. The transistor is the atomic switch of digital electronics: apply a voltage to its gate and it conducts, remove it and it blocks. Wire millions of these switches in the right pattern and you get adders, processors, memories and radios.

The word "scale" refers to how many devices share one chip. Integration grew through clear eras — SSI, MSI, LSI and finally VLSI — as photolithography printed ever-smaller features. A 1971 microprocessor held ~2,300 transistors; modern processors hold tens of billions on a fingernail-sized die. No human places those by hand: we describe behaviour in Verilog and let synthesis build the gates. RTL is therefore the human entry point into VLSI — the purpose of this module.

What · Why · How · Limitation

What: a complete system on silicon. **Why:** integration cuts cost, power and size while raising performance. **How:** lithography prints transistors; designers describe behaviour in HDL. **Limitation:** physics (heat, leakage, cost) is slowing pure scaling — so efficient design and reuse now differentiate.

Key takeaways

- VLSI = an entire system of millions–billions of transistors on one die.
- Integration scaled SSI → MSI → LSI → VLSI as feature sizes shrank.
- RTL is how humans design at that scale; the tools build the transistors.

Interview & industry

Q: What does the "scale" in VLSI refer to, and name the integration eras.

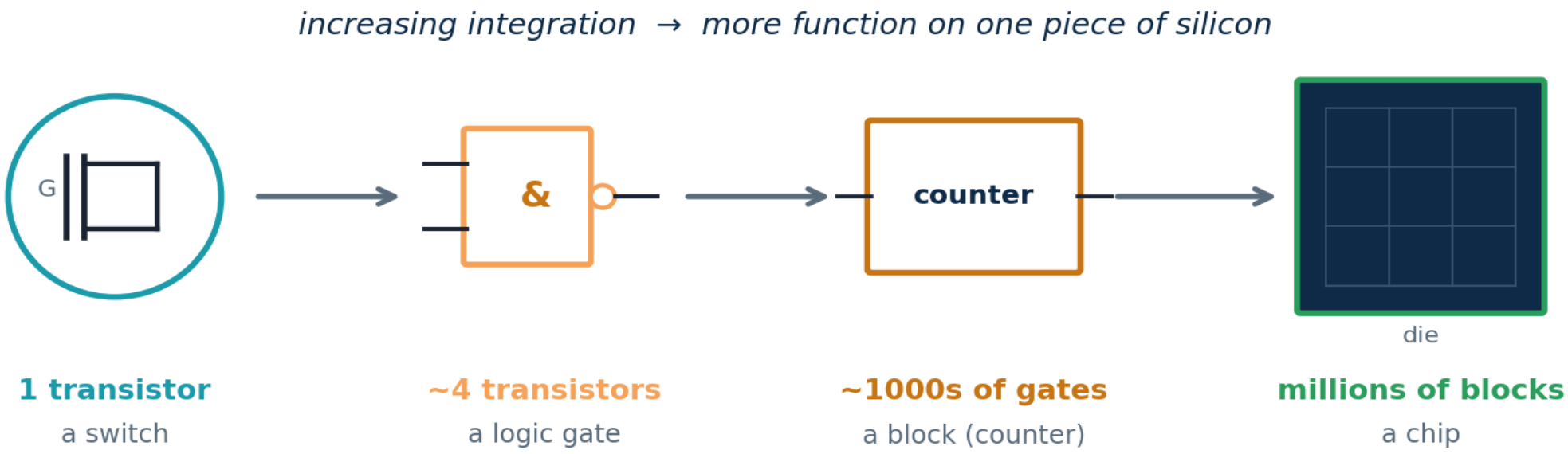
Tip: Know approximate transistor counts per era; it is a common warm-up question.

Running example used throughout this topic

We will follow one tiny design — a **4-bit binary counter** — from idea, through the design flow, to simulation and synthesis in the tutorials. Reusing one example keeps every concept concrete and connected.

From a Transistor to a System — Integration by Hierarchy

How do a few switches become a billion-transistor chip? By hierarchy. Each level is built from many copies of the level below: transistors form logic gates, gates form functional blocks, and blocks tile up into a complete chip. You, the RTL designer, work at the block level — describing behaviour — while synthesis and the foundry handle the gates and transistors beneath you.



Why this matters: abstraction is what makes VLSI humanly possible. Nobody reasons about a billion transistors at once; they reason about a handful of blocks, each verified and reused. RTL is the level where that human reasoning happens.

TOPIC 1A · TIERED DEPTH

Understanding VLSI at Four Levels

The same idea, deepened. Every concept in this module is taught at four levels so it is accessible to a beginner yet useful to a practitioner.

BASIC

VLSI means putting a whole circuit — lots of tiny switches (transistors) — onto one small chip instead of many separate parts.

INTERMEDIATE

Those switches form logic gates; gates form blocks (adders, registers); blocks form systems. Designers work at the block level using a hardware language, not at the transistor level.

ADVANCED

Designs are captured at register-transfer level (RTL) and synthesised to gates for a target process. Trade-offs in area, timing and power are managed quantitatively across the flow.

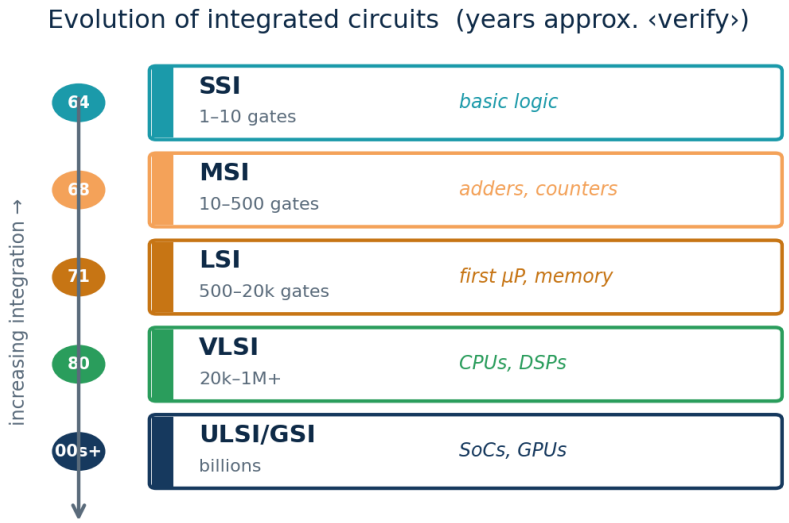
INDUSTRY

With scaling slowing, value comes from architecture, verified reusable IP, low-power design and fast timing closure — so clean, synthesis-aware, well-verified RTL is the skill employers pay for.

TOPIC 1A · EVOLUTION

The Evolution of Integrated Circuits

Before ICs, computers used discrete transistors and vacuum tubes — bulky, hot, unreliable. Jack Kilby (TI, 1958) and Robert Noyce (Fairchild, 1959) independently integrated components on one substrate, founding the IC. Integration density then climbed through well-defined generations, each multiplying gates per chip and unlocking new product classes.



Each era enabled a new product class

SSI (1–10 gates): basic logic on a chip → early digital boards.

MSI (tens of gates): adders, counters, multiplexers → calculators.

LSI (hundreds–thousands): the first microprocessors and memories.

VLSI (tens of thousands+): full CPUs, DSPs, graphics.

ULSI / GSI (millions–billions): today's SoCs, GPUs, AI chips.

Gate-count bands are approximate ‹verify exact ranges›.

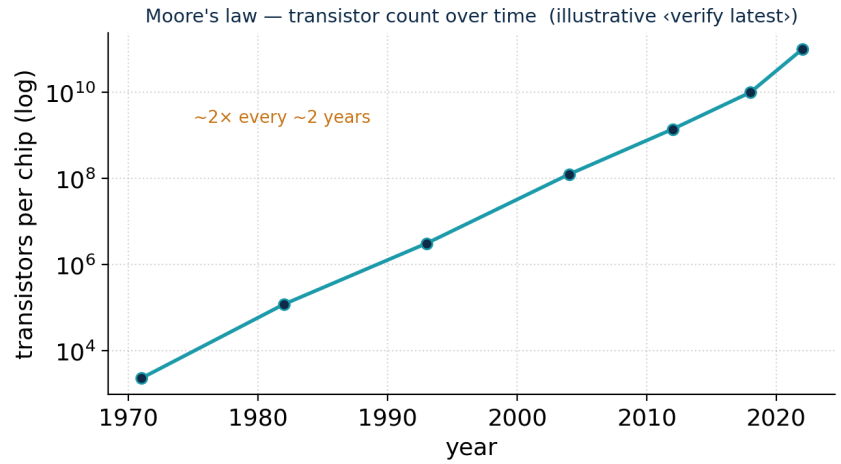
TOPIC 1A · TREND

Moore's Law and Why Scaling Is Slowing

Moore's law (Gordon Moore, 1965) observes that transistors per chip double roughly every two years at constant cost. It is an economic/historical trend, not a physical law — but it held for ~50 years and drove the digital revolution by making computing cheaper every cycle.

Dennard scaling was the companion: as transistors shrank, power density stayed constant, so smaller also meant faster and cooler. It broke down around 2005 — shrinking no longer cut power proportionally — so clock speeds plateaued and the industry pivoted to multi-core and accelerators.

Today geometric scaling is decelerating and per-transistor cost has stopped falling, so performance now comes from architecture (chiplets, 3D stacking, AI accelerators) and design efficiency — raising the value of disciplined RTL.



Key takeaways

- Moore's law: ~2× transistors every ~2 years — economic, not physical.
- Dennard scaling (constant power density) broke ~2005 → multi-core & accelerators.
- Scaling is slowing; architecture, reuse and good RTL are the new levers.

Interview & industry

Q: Difference between Moore's law and Dennard scaling?

Tip: Quote latest node/transistor figures from a current source, marked ‹verify› — never from memory.

TOPIC 1A · PRACTICAL EXAMPLE 1

Numerical: Applying Moore's Law

Problem

The Intel 4004 (1971) had about **2,300 transistors**. If transistor count doubles every 2 years, estimate the count 50 years later (2021), and compare with reality.

Step 1 — number of doublings:

n = 50 years ÷ 2 = 25 doublings

Step 2 — apply the doubling:

$N = 2{,}300 \times 2^{25} = 2{,}300 \times 33{,}554{,}432$

$N \approx 7.7 \times 10^{10}$ **≈ 77 billion transistors**

Interpretation

Real high-end chips around 2021 held **tens of billions** of transistors ‹verify exact figures› — the same order of magnitude as our estimate.

The model slightly overshoots because the doubling period stretched toward ~2.5–3 years as scaling slowed.

Lesson: Moore's law is a remarkably good first-order estimator over decades, but it is a trend — always sanity-check against current data, and treat memorised figures as ‹verify›.

Try it yourself

If a design had 1 million gates in 2010, roughly how many would the trend predict for 2020? (Answer: 5 doublings → $\times 2^5 = \times 32 \approx 32$ million gates.) Discuss why the real number is usually lower.

TOPIC 1A · APPLICATIONS

Where VLSI Chips Are Used — Six Domains

VLSI is invisible but everywhere. The same RTL skills serve every domain, each with a dominant constraint — performance, power, cost, safety or reliability — that shapes its sign-off criteria.

Compute & data centre

CPUs, GPUs, AI accelerators; performance & memory bandwidth dominate, power/cooling limit.

Mobile & consumer

Phone/wearable SoCs, TVs; energy-per-operation (battery life) is king.

Automotive

ECUs, ADAS, EV power; functional safety (ISO 26262) and temperature range dominate.

Medical

Imaging, implants, monitors; ultra-low power, reliability, certification.

Industrial / IoT

Sensors, controllers, edge AI; cost, integration, long lifetimes.

Space & defence

Radiation-hardened, high-reliability parts; correctness over raw speed.

Same skill, different sign-off: a counter written in Verilog is the same RTL in a toy or a satellite — the satellite just adds radiation hardening and far heavier verification.

TOPIC 1A · INDUSTRY STRUCTURE

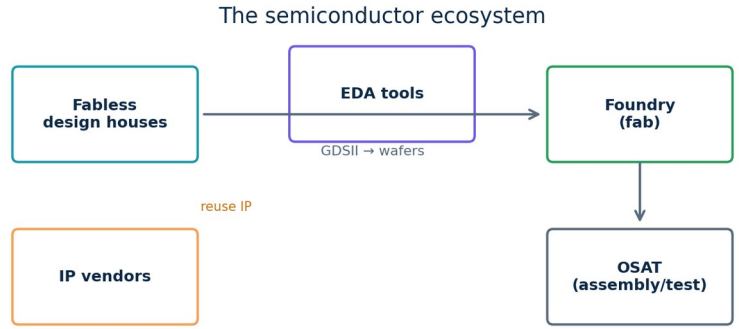
The Semiconductor Ecosystem

No single company designs, makes, packages and sells a chip alone. The industry specialised, and the chain tells you where an RTL engineer works.

Fabless design houses create the chip — RTL is written and verified here — but own no factory.

Foundries manufacture silicon from the GDSII the design house sends; a leading-edge fab costs tens of billions.

IP vendors license reusable blocks; EDA companies provide the tools; OSATs assemble, package and test.



Key takeaways

- Fabless designs (RTL here) → foundry manufactures → OSAT packages/tests.
- IP vendors supply reusable blocks; EDA vendors supply the tools.
- Specialisation gave each layer huge scale and capital efficiency.

Interview & industry

Q: What is a fabless company, and why did the model win?

Tip: India's Semiconductor Mission funds across the chain; near-term graduate opportunity is design (RTL/verification).

Where you fit: this module trains the front-end design skill fabless houses and IP vendors hire for — your Verilog becomes the foundry's input and, ultimately, silicon.

ASIC vs FPGA vs SoC — Choosing the Silicon

RTL is the common starting point, but it can be realised on different silicon. The choice trades upfront cost against unit cost, performance and time-to-market — and the same Verilog can target all three.

ASIC vs FPGA vs SoC

	ASIC	FPGA	SoC
Nature	custom silicon	reconfigurable	system on one chip
Time to market	long	short	medium
Unit cost (volume)	lowest	higher	low–medium
NRE / upfront cost	very high	low	high
Performance / power	best	good	application-tuned
Best for	high volume	prototyping, low volume	integrated products

The RTL connection

Same synthesizable Verilog targets all three.

FPGA — reprogrammable, low upfront cost: prototype and low volume.

ASIC — custom silicon, lowest unit cost and best power at high volume, but huge NRE.

SoC — a whole system (logic + CPU + IP) on one die; the dominant modern form.

TOPIC 1A · PRACTICAL EXAMPLE 2

Numerical: ASIC vs FPGA Break-Even

Problem & setup

You must ship a product. Two options (illustrative ‹verify›):

FPGA: NRE = $0, unit cost = $50.

ASIC: NRE = $2,000,000, unit cost = $5.

Find the volume N where total costs are equal.

Total cost = NRE + unit×N:

FPGA: 50N ASIC: 2,000,000 + 5N

Set equal: 50N = 2,000,000 + 5N

45N = 2,000,000 → N ≈ 44,444 units

Decision

Below ~44,000 units the FPGA is cheaper overall — no NRE to amortise.

Above ~44,000 units the ASIC wins — its low unit cost repays the NRE.

This is why teams **prototype on FPGA** and migrate to ASIC only once volume justifies the NRE.

Real decisions also weigh power, performance, time-to-market and risk — not cost alone — but the break-even is the first cut.

Try it yourself

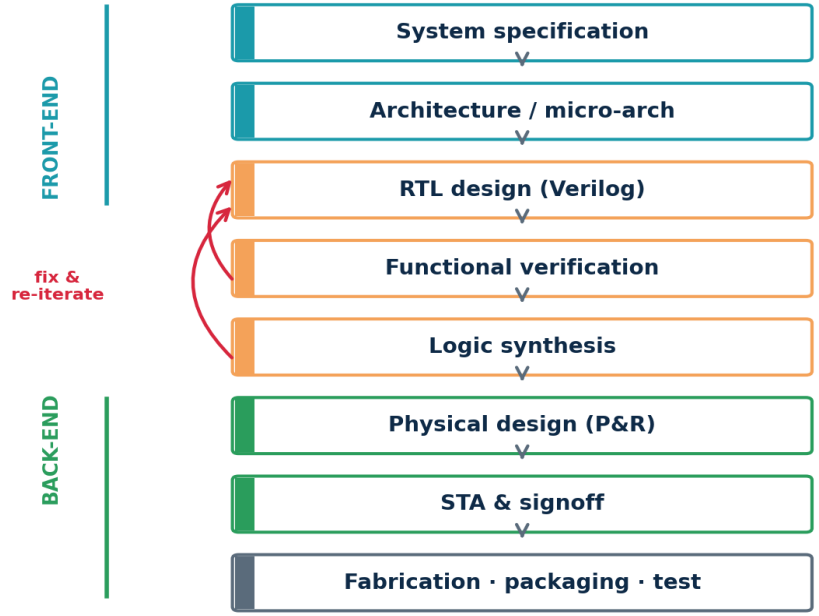
If ASIC NRE were $500,000 instead, what is the new break-even? (45N = 500,000 → N ≈ 11,111 units — a lower NRE makes the ASIC attractive much sooner.)

The VLSI Design Flow — Specification to Silicon

A chip is built through a disciplined sequence, each stage refining the design and handing it on.

1. **Specification & architecture** — define function, performance, power, area; partition into blocks and a micro-architecture.
2. **RTL design** — capture behaviour in synthesizable Verilog. The creative heart of the front-end and the focus of this module.
3. **Functional verification** — prove the RTL matches the spec via testbenches, simulation, assertions and coverage, before any silicon.
4. **Logic synthesis** — map RTL to a gate-level netlist for a target library, optimising timing, area and power.
5. **Physical design** — floorplan, place, route and build the clock tree into a layout.
6. **STA, signoff, fabrication** — verify timing across corners, run signoff, send GDSII to the foundry for manufacture, packaging and test.

The VLSI design flow (with iteration)



Animation: reveal stages top-down; highlight RTL + verification (this module); fade the back-end.

TOPIC 1B · TIERED DEPTH

Understanding the Design Flow at Four Levels

The design flow, deepened from a beginner's mental model to how it is run in industry.

BASIC

You decide what the chip should do, describe it, check it works, then turn it into something a factory can build.

INTERMEDIATE

Spec → RTL (Verilog) → verify by simulation → synthesise to gates → place & route → send layout (GDSII) to the foundry.

ADVANCED

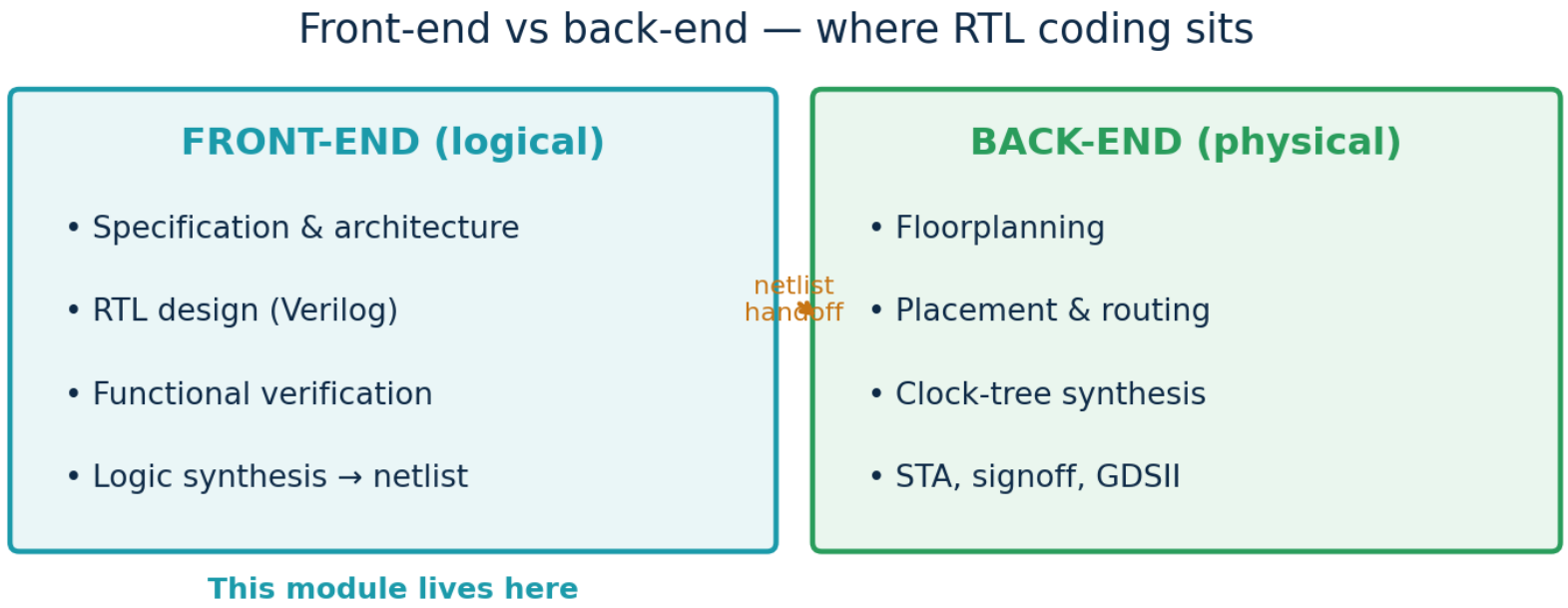
Each stage has tools and checks: lint and coverage in verification; timing/area/power optimisation in synthesis; STA across process corners before signoff. The flow iterates — failures loop back to RTL.

INDUSTRY

Front-end and back-end teams hand off a constrained netlist; 'shift-left' pushes verification early because a silicon respin costs months and millions. Clean, synthesis-aware RTL minimises late surprises.

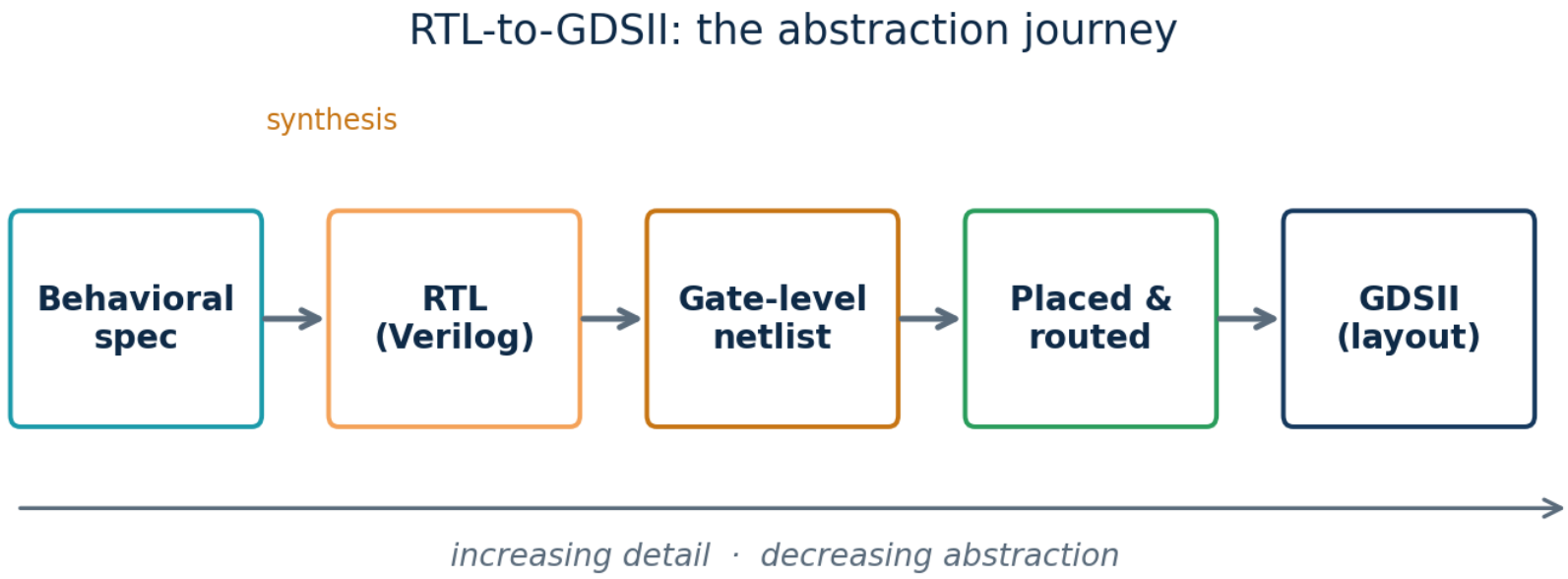
Front-End vs Back-End — Where RTL Lives

The flow divides into a **logical front-end** (what the chip does) and a **physical back-end** (how it is laid out). They meet at the gate-level netlist handoff. RTL engineers own the front-end — but must write code aware of the back-end's timing and area realities.



RTL-to-GDSII — Your Code Becomes Polygons

From intent to manufacturable layout is a steady descent in abstraction: you write behaviour at RTL; synthesis makes a gate-level netlist; place-and-route makes a physical arrangement; GDSII is the polygon file the foundry uses. You will perform the RTL→netlist step yourself in Tutorial T2.



Why care as an RTL coder? Every downstream stage inherits your decisions. Huge combinational paths, accidental latches or a poor reset strategy create timing/area problems the back-end cannot fully fix. Clean RTL is the cheapest place to get the whole chip right.

TOPIC 1B · WORKED EXAMPLE

Tracing a 4-Bit Counter Through the Flow

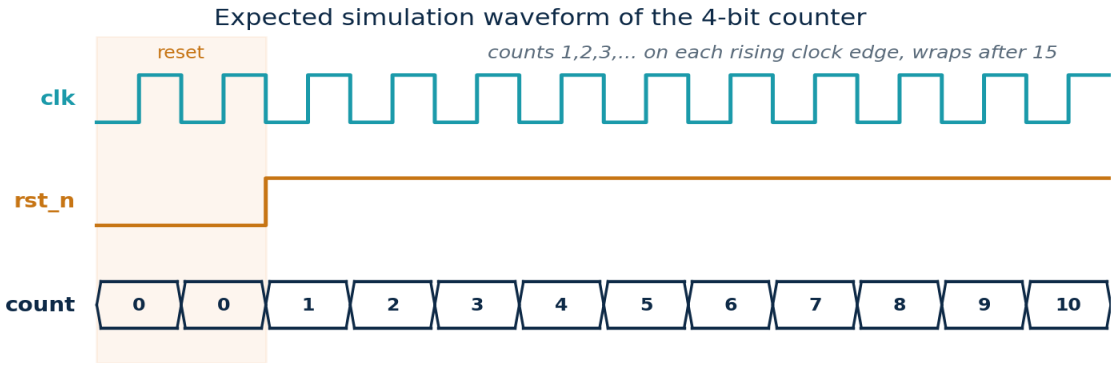
Our running example, stage by stage — so the abstract flow becomes concrete before we ever write full Verilog.

1 · Spec	“A 4-bit counter: on each rising clock edge, increment 0→15 then wrap; an active-low reset forces 0.”
2 · RTL	A short Verilog module with a clocked always block (shown in Tutorial T2). Human-readable behaviour.
3 · Verify	A testbench drives clk and rst_n and checks the count sequence; we view it as a waveform in GTKWave.
4 · Synthesis	Yosys maps the RTL to gates: four flip-flops + an incrementer. 'stat' reports the cell/area count.
5 · Phys/GDSII	Place-and-route would arrange those cells and wires into a layout (out of scope here; done by OpenLane).

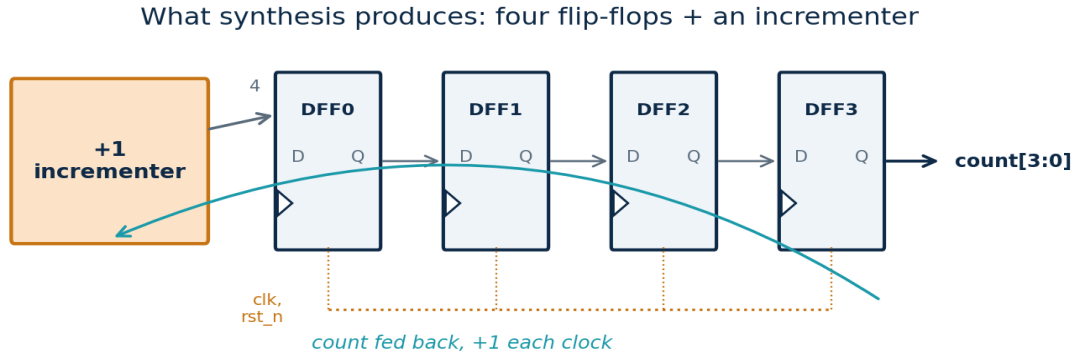
What the Counter Becomes — Two Views of One Design

The same counter has two complementary views. Verification looks at behaviour over time (a waveform); synthesis looks at structure (gates). Both describe the identical design — you will produce both in Tutorial T2.

Behaviour (what verification sees)



Structure (what synthesis produces)



Reading the waveform

While `rst_n=0` the count is held at 0. Once reset releases, `count` increments by 1 on every rising clock edge — 0,1,2,...,15 — then wraps back to 0. This is the functional behaviour a testbench checks.

Reading the gates

Synthesis turns the RTL into **four D flip-flops** (one per bit) plus an **incrementer** that adds 1 to the current value and feeds it back. Yosys' `stat` report will list exactly these cells — structure, not behaviour.

TOPIC 1B · TUTORIAL T1

Set Up the Open-Source RTL Toolchain

Objective: install and verify the free toolchain on Ubuntu so every later lab runs. Industry relevance: these tools mirror commercial EDA steps (simulate, synthesize, analyse).

```
# Ubuntu terminal  (‹verify versions›)

# 1. update package lists
sudo apt update

# 2. install simulator, waveform viewer,
#    linter, synthesis tool, git
sudo apt install -y iverilog gtkwave \
     verilator yosys git

# 3. verify each tool is on the PATH
iverilog -V
yosys -V
gtkwave --version
verilator --version
```

Notes & troubleshooting

Recommended alternative: the OSS CAD Suite bundle ships current, matched versions of all these tools — prefer it if the apt versions are old ‹verify›.

'command not found' → the install failed or the PATH is stale; re-run install, reopen the terminal.

GTKWave won't open on a headless server → use an X-forwarding SSH session or a desktop VM.

Expected output: each '-V' prints a version banner. That confirms a working environment.

Checkpoint: all four version commands print a banner with no errors. Commit a short README to Git recording the versions you installed — reproducibility is an industry habit.

TOPIC 1B · TUTORIAL T2

Run the Counter: Compile → Simulate → View → Synthesize

Objective: take the running 4-bit counter through the front-end flow on real tools. (Full Verilog is taught in Topic 4; here you run it to see the flow work.)

counter4.v (synthesizable design)

```
module counter4 (
  input            clk,
  input            rst_n,   // active-low
  output reg [3:0] count
);
  always @(posedge clk or negedge rst_n)
    if (!rst_n) count <= 4'b0000;
    else        count <= count + 1'b1;
endmodule
```

counter4_tb.v (testbench – sim only)

```
`timescale 1ns/1ps
module counter4_tb;
  reg clk=0, rst_n=0; wire [3:0] count;
  counter4 dut(.clk(clk),.rst_n(rst_n),
               .count(count));
  always #5 clk = ~clk;      // 100 MHz
  initial begin
    $dumpfile("c.vcd"); $dumpvars(0,counter4_tb);
    #12 rst_n=1; #200 $finish;
  end
endmodule
```

 # simulate + view waveform

```
iverilog -o c.vvp counter4_tb.v counter4.v
vvp c.vvp
gtkwave c.vcd &
# expect: count 0,1,2,...,15,0,... after reset
```

 # synthesize with Yosys

```
yosys -p "read_verilog counter4.v; \
  synth -top counter4; stat"
# expect: ~4 DFFs + an incrementer;
# 'stat' lists the cell/area summary
```

Guardrail: $dumpfile/$dumpvars and #delays live only in the testbench — never in the synthesizable design. This is the rule we enforce all module.

TOPIC 1 · GLOSSARY

Key Terms & Acronyms (Topic 1)

VLSI	Very-Large-Scale Integration — millions+ transistors on one chip
IC / die	Integrated Circuit / the silicon piece a chip is cut from
RTL	Register-Transfer Level — the design-entry abstraction
HDL	Hardware Description Language (e.g., Verilog)
Synthesis	mapping RTL to a gate-level netlist
Netlist	a circuit described as gates and their connections
GDSII	the layout/polygon file sent to the foundry
ASIC	Application-Specific IC — custom silicon
FPGA	Field-Programmable Gate Array — reconfigurable chip
SoC	System-on-Chip — logic + CPU + IP on one die
Fabless / Foundry	design house without a fab / the manufacturer
NRE	Non-Recurring Engineering — one-time design/tooling cost
STA	Static Timing Analysis
Tapeout	sending the final layout to the foundry

TOPIC 1 · RECAP

Topic 1 — Consolidation & Self-Check

What you should now be able to do

Explain what VLSI is and why RTL is the human entry point (Basic→Industry).

Recount the IC eras and state Moore's law and why scaling is slowing.

Apply Moore's law and an ASIC/FPGA break-even numerically.

Identify application domains, the ecosystem, and ASIC/FPGA/SoC trade-offs.

Trace the design flow and a 4-bit counter through it.

Run the toolchain: install it (T1) and compile/simulate/synthesize (T2).

Self-check

- Why can't a billion-transistor chip be drawn gate by gate?
- State Moore's law and one reason scaling is slowing.
- Fabless vs foundry — what is the difference?
- At what volume does the ASIC beat the FPGA in our example?
- What does synthesis produce, and which tool did we use?
- Which file in T2 is NOT synthesizable, and why?

Looking ahead

Topic 2 — RTL Design Methodology zooms into the front-end: what RTL really is, why it is the design-entry level, and the HDL landscape — setting up the hands-on Verilog coding in Topic 4, still using our counter.